



ŽILINSKÁ UNIVERZITA V ŽILINE

Fakulta riadenia
a informatiky

Semestrálna práca z predmetu
vývoj aplikácií pre mobilné zariadenia

ŽILINA GUIDE

Vypracoval: Illia Ivanilov

Študijná skupina: 5ZYI35

Akademický rok: 2025/2026

V Žiline dňa 9.5.2026



Obsah

Úvod.....	2
Prehľad podobných aplikácií	2
1. Google Maps	2
2. Mapy.com	3
3. Yelp	4
Analýza navrhovanej aplikácie	5
Používateľské roly	5
Prípady použitia	5
Mimofunkčné požiadavky	5
Návrh architektúry aplikácie	5
Dátový model.....	5
Hlavné procesy.....	6
Architektúra aplikácie	6
Popis vrstiev.....	7
Návrh vzhľadu obrazoviek.....	7
Mapa.....	8
Zoznam	8
Detail.....	10
Obľúbené	11
6. Popis implementácie.....	12
6.1 Obrazovky.....	12
6.2 ViewModel a Lifecycle	12
6.3 Navigation.....	12
6.4 WorkManager a notifikácie.....	12
6.5 Sieťová komunikácia - Retrofit a Overpass API	12
6.6 GPS senzor	12
6.7 DataStore - Obľúbené miesta	13
6.8 OSMDroid - Mapa	13
6.9 Zachovávanie stavu pri rotácii	13
6.10 Architektúra a kvalita kódu	13
7. Zoznam použitých zdrojov	13

Úvod

Cieľom tejto semestrálnej práce je návrh a implementácia mobilnej aplikácie Žilina Guide pre operačný systém Android. Aplikácia slúži ako interaktívny sprievodca po meste Žilina, ktorý umožňuje používateľom objavovať zaujímavé miesta prostredníctvom interaktívnej mapy a zoznamu bodov záujmu rozdelených do kategórií.

Hlavným motívom vzniku aplikácie je snaha poskytnúť návštevníkom aj obyvateľom Žiliny jednoduchý a prehľadný nástroj na objavovanie mesta. Existujúce riešenia ako Google Maps sú univerzálne a neposkytujú zážitok prispôsobený konkrétnemu mestu. Údaje sú získavané z verejného rozhrania Overpass API (OpenStreetMap), obľúbené miesta si používateľ ukladá lokálne pomocou DataStore.

Aplikácia je implementovaná v programovacom jazyku Kotlin s používateľským rozhraním Jetpack Compose a dodržiava princípy čistej architektúry (Clean Architecture) s návrhom MVVM. Využíva knižnice a komponenty ekosystému Jetpack — Navigation, ViewModel, Lifecycle, WorkManager a ďalšie.

Hlavné ciele práce sú:

- Navrhnuť a implementovať prehľadnú a intuitívnu mobilnú aplikáciu pre platformu Android
- Integrovať interaktívnu mapu s reálnymi údajmi z OpenStreetMap
- Zabezpečiť správnu reakciu aplikácie na zmenu orientácie displeja
- Aplikovať princípy čistého kódu, objektovo orientovaného programovania a modernej Android architektúry

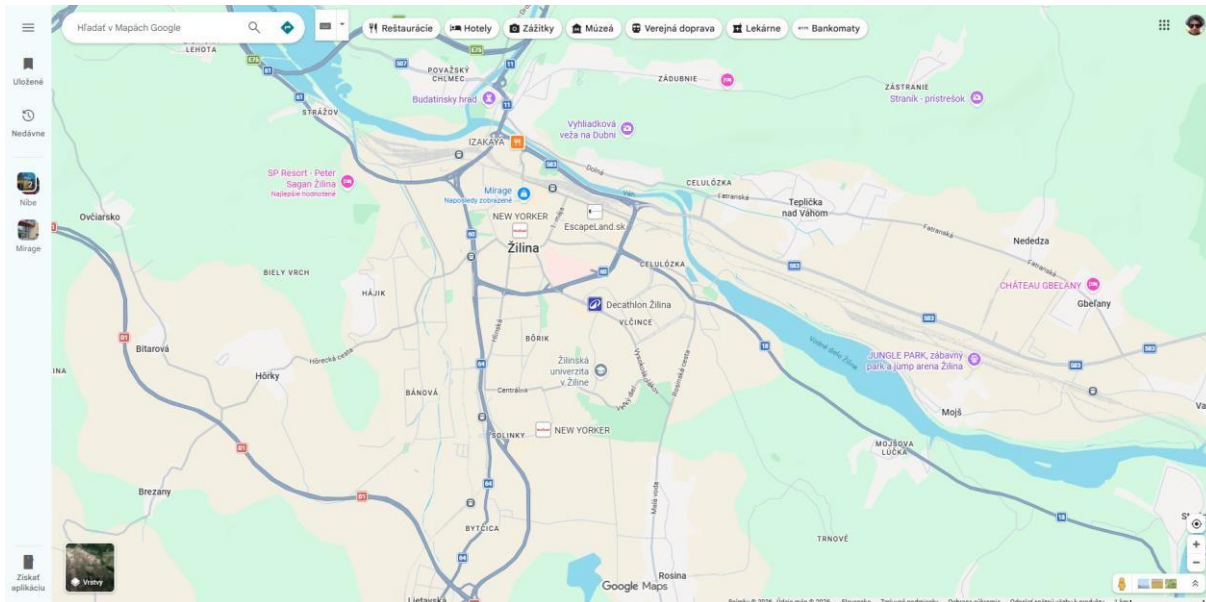
Prehľad podobných aplikácií

1. Google Maps

Google Maps je najrozšírenejšia navigačná a mapová aplikácia na svete. Umožňuje vyhľadávanie miest, navigáciu, zobrazenie recenzií a otváracích hodín. Obsahuje obrovské množstvo POI dát pre celý svet vrátane Žiliny.

Výhody: komplexné dáta, offline mapy, navigácia, recenzie

Nevýhody: univerzálnosť - nie je prispôbena konkrétnemu mestu, zložité rozhranie, vyžaduje Google účet





2. Mapy.com

Mapy.com je nástupca portálu Mapy.cz od spoločnosti Seznam.cz. Ponúka podrobné vyhľadávanie na mapách celého sveta, zobrazenie turistických chodníkov, cyklotrás a plánovanie trás. Aplikácia podporuje offline mapy celého sveta, navigáciu, plánovač trás a sledovanie aktivít. Offline režim funguje plnohodnotne vrátane navigácie a vyhľadávania.

Výhody: kvalitné dáta pre SR, offline režim, turistické a cyklomapy, bez poplatkov

Nevýhody: nie je zameraná na konkrétne mesto, rozhranie je komplexné a pre bežného mestského používateľa zbytočne rozsiahle, niektoré pokročilé funkcie sú dostupné len v platenej verzii Premium



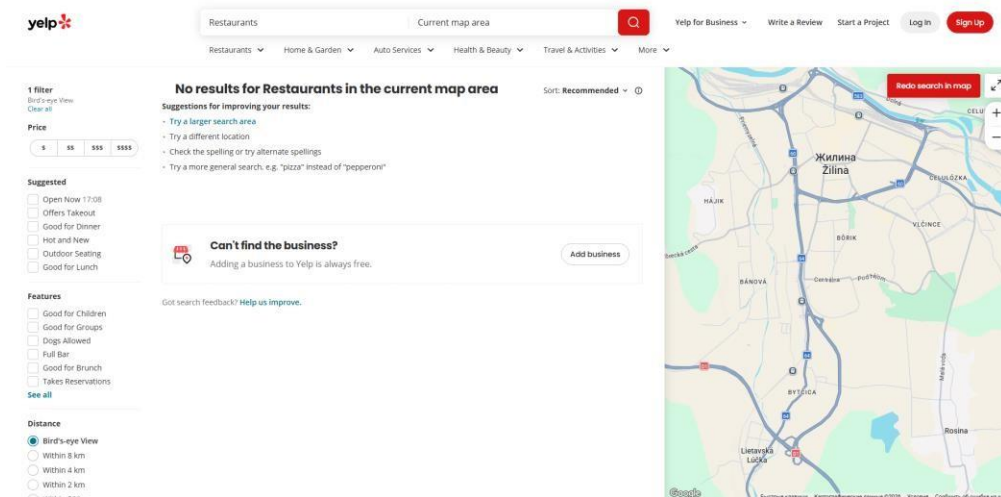


3. Yelp

Aplikácia zameraná na objavovanie podnikov - reštaurácií, kaviarní, zábavy. Obsahuje recenzie, fotografie, otváracie hodiny a filtrovanie podľa kategórií.

Výhody: detailné informácie o podnikoch, používateľské recenzie, filtrovanie

Nevýhody: slabé pokrytie pre Žilinu a SR všeobecne, zameraná výhradne na podniky



Analyzované aplikácie sú univerzálne nástroje pokrývajúce celý svet, čo sa odráža na ich komplexnosti a menej intuitívnom rozhraní pre bežného používateľa hľadajúceho miesta v konkrétnom meste. Aplikácia Žilina Guide sa odlišuje cieľným zameraním výhradne na Žilinu, jednoduchým a prehľadným rozhraním bez nutnosti registrácie a prispôbenou kategorizáciou miest relevantnou pre návštevníkov mesta.

Analýza navrhovanej aplikácie

Používateľské roly

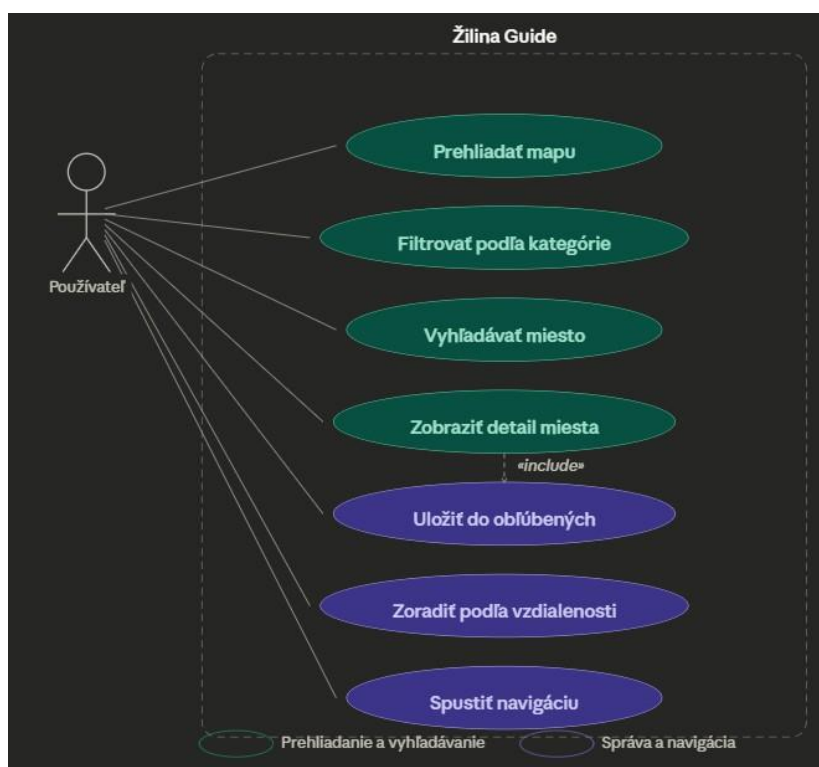
Aplikácia Žilina Guide má jedinú používateľskú rolu - bežný používateľ (návštevník alebo obyvateľ mesta). Aplikácia nevyžaduje registráciu ani prihlásenie, nevyžaduje administrátorský prístup a všetky funkcie sú dostupné každému bez obmedzení.

Prípady použitia

Používateľ môže prehliadať interaktívnu mapu Žiliny s bodmi záujmu, filtrovať zobrazené miesta podľa kategórie, vyhľadávať konkrétne miesto podľa názvu, zobraziť detail vybraného miesta, uložiť miesto do obľúbených alebo ho z obľúbených odstrániť, zoradiť zoznam miest podľa vzdialenosti od aktuálnej polohy a spustiť navigáciu do vybraného miesta prostredníctvom externej aplikácie.

Mimofunkčné požiadavky

Aplikácia musí správne reagovať na zmenu orientácie displeja (portrait/landscape) bez straty stavu. Musí byť funkčná aj bez internetového pripojenia v obmedzenom režime s využitím záložných dát. Odozva UI nesmie blokovať hlavné vlákno - všetky sieťové operácie prebiehajú asynchrónne. Aplikácia je kompatibilná s Android 8.0 (API 26) a vyšším.



Návrh architektúry aplikácie

Dátový model

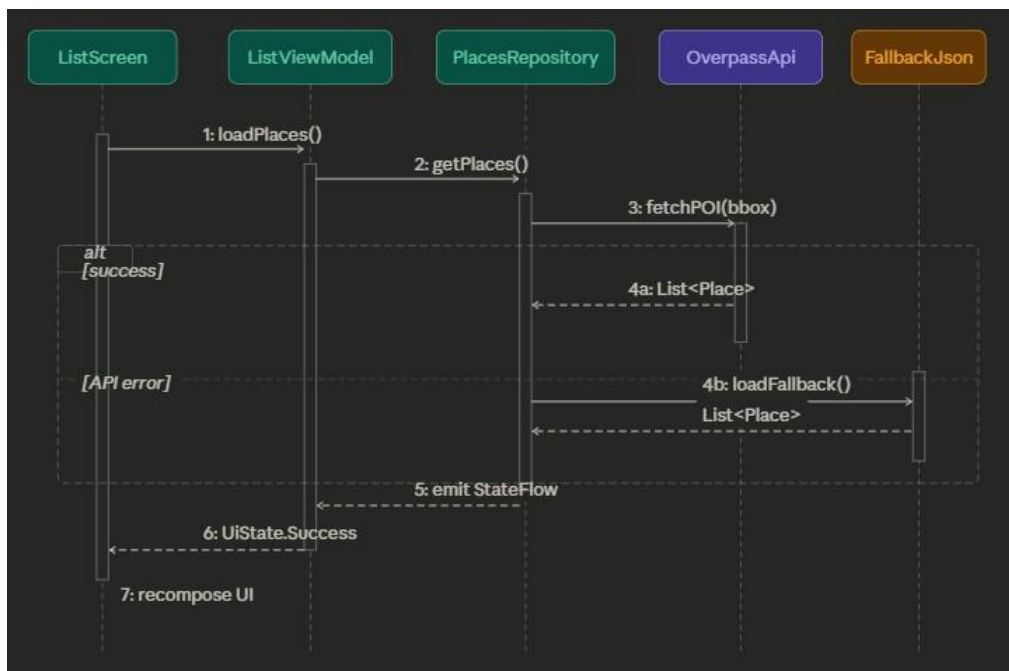
Jadrom aplikácie je entita Place, ktorá reprezentuje jeden bod záujmu. Každé miesto má jednoznačný identifikátor, názov, geografické súradnice, kategóriu, popis, URL adresu obrázka a voliteľné otváracie hodiny. Kategória je reprezentovaná enumeráciou Category s hodnotami CAFE, RESTAURANT, ATTRACTION, PARK a SHOP. Obľúbené miesta sú

uložené ako množina identifikátorov v DataStore - teda nie ako samostatná entita, ale ako jednoduchý zoznam referencií na existujúce miesta.



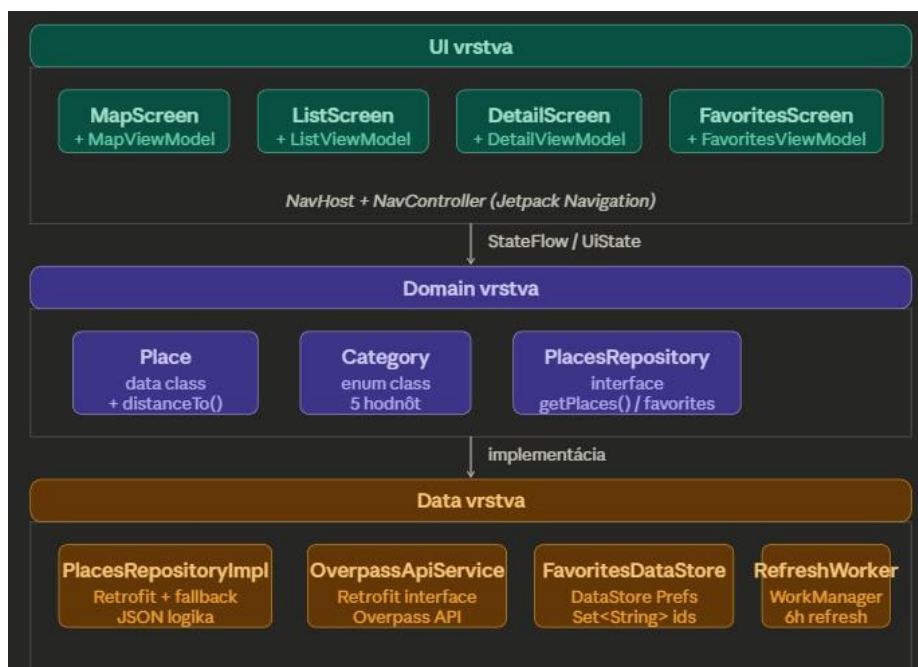
Hlavné procesy

Nasledujúci diagram zobrazuje sekvenciu udalostí pri načítaní zoznamu miest po spustení aplikácie. ViewModel požiada repozitár o dáta, repozitár sa pokúsi zavolať Overpass API. Pri neúspešnom volaní (napr. offline stav) sa použijú záložné dáta zo statického JSON súboru v resources.



Architektúra aplikácie

Aplikácia je navrhnutá podľa vzoru MVVM (Model-View-ViewModel) s vrstvením Clean Architecture. Rozdelená je do troch hlavných vrstiev - UI, Domain a Data - ktoré sú od seba striktné oddelené. Každá vrstva komunikuje len so susednou vrstvou a závislosť smeruje vždy smerom dnu (UI => Domain => Data), nikdy naopak.



Popis vrstiev

UI vrstva

Obsahuje Compose obrazovky a ich ViewModely. ViewModel drží stav obrazovky ako `StateFlow<UiState>` a reaguje na používateľské akcie volaním repozitára. Obrazovky sú pasívne — len pozorujú stav a vykresľujú používateľské rozhranie.

Dátová vrstva

Obsahuje konkrétnu implementáciu repozitára (`PlacesRepositoryImpl`), rozhranie Retrofit pre Overpass API, triedu `FavoritesDataStore` na perzistovanie obľúbených miest a triedu `RefreshWorker` pre periodické obnovovanie dát na pozadí.

Návrh vzhľadu obrazoviek

Aplikácia je postavená na štandardných komponentoch Material 3 a Jetpack Compose. Navigáciu medzi obrazovkami zabezpečuje spodná lišta s ikonami, každá obrazovka má vlastnú hornú lištu s názvom a prípadnými akciami. Na vyhľadávanie slúži textové pole, miesta je možné filtrovať pomocou farebných čipov pre každú kategóriu. Zoznam miest je zobrazený ako plynulo scrollovateľný stĺpec kartičiek, pričom každá kartička zobrazuje fotografiu načítanú zo siete, názov a základné informácie. Mapa je integrovaná ako natívny Android view s OSMDroid knižnicou a obsahuje plávajúce tlačidlo pre vycentrovanie na polohu používateľa. Obľúbené miesta je možné pridávať a odoberať jedným kliknutím na ikonu srdiečka. Stavové hlásenia, napríklad o chybe pri načítaní dát, sa zobrazujú ako nenápadný panel v spodnej časti obrazovky.

Mapa



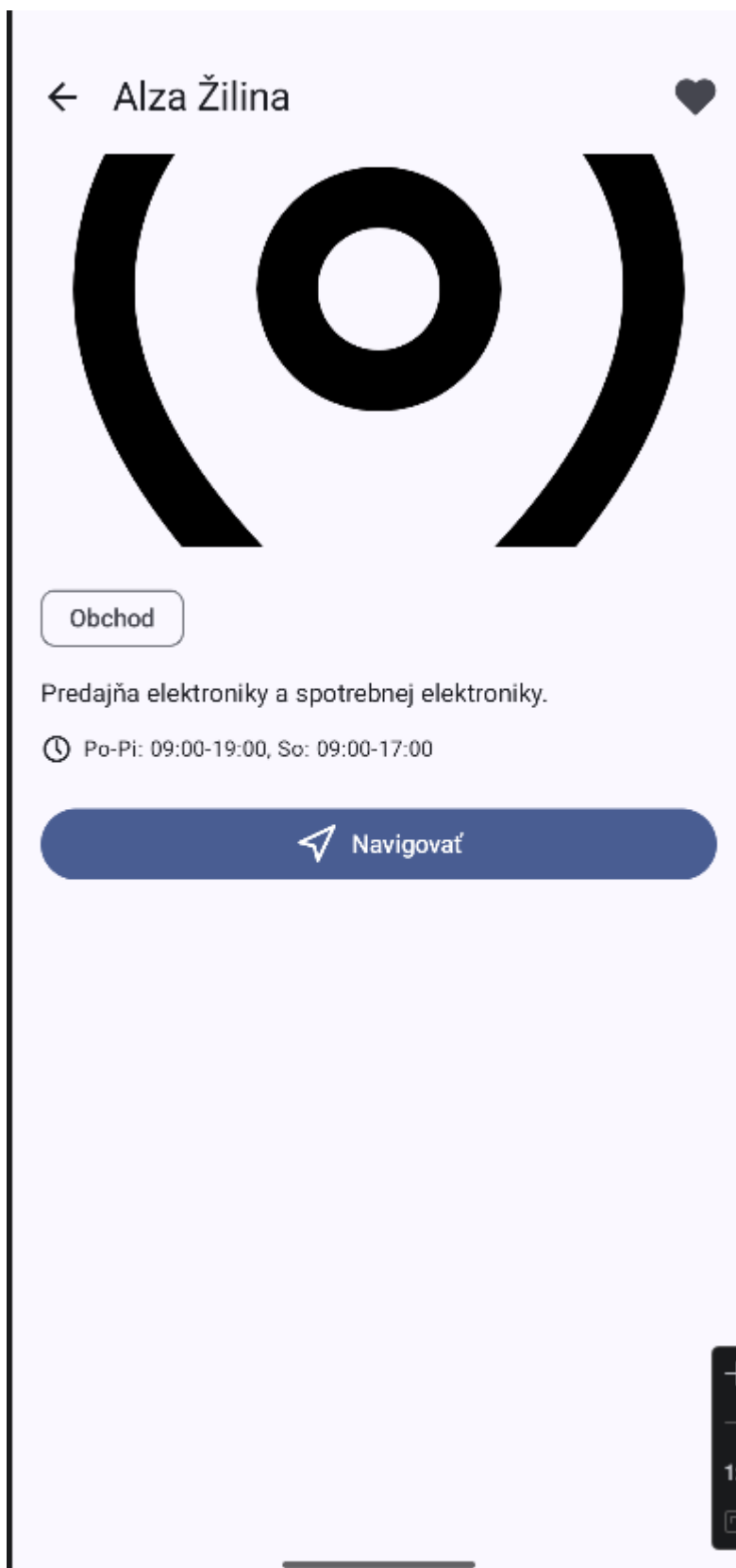
Obrazovka mapy je hlavnou obrazovkou aplikácie. Zobrazuje OSMDroid mapu vycentrovanú na Žilinu s markermi. Po kliknutí na marker sa zobrazí detail. V hornej časti sú FilterChip komponenty pre filtrovanie kategórií a FloatingActionButton slúži na vycentrovanie mapy na GPS polohu používateľa.

Zoznam



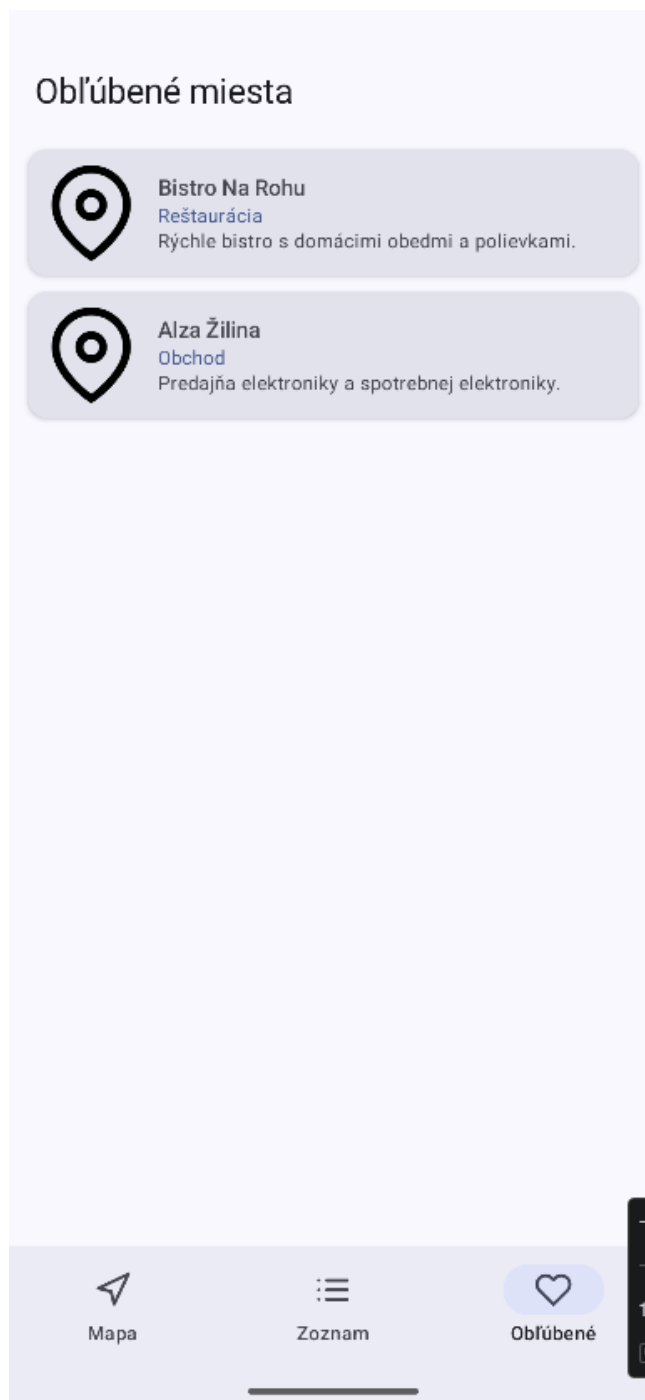
Obrazovka zoznamu zobrazuje všetky miesta ako scrollovateľný LazyColumn. V hornej časti je SearchBar pre vyhľadávanie podľa názvu a FilterChip lišta. Každá kartička zobrazuje ikonu kategórie, názov, adresu a vzdialenosť od aktuálnej polohy.

Detail



Obrazovka detailu sa otvorí po kliknutí na konkrétne miesto. Obsahuje fotografiu načítanú cez Coil, kategóriu v badge komponente, adresu, otváracie hodiny, popis a tlačidlo „Navigovať“, ktoré spustí externú navigačnú aplikáciu. V pravom hornom rohu je IconButton so srdiečkom pre uloženie do obľúbených.

Obľúbené





Obrazovka obľúbených zobrazuje miesta uložené používateľom. Dáta sú perzistované cez DataStore Preferences - ukladajú sa len identifikátory miest. Klik na položku otvorí detail, klik na srdiečko miesto odoberie.

6. Popis implementácie

6.1 Obrazovky

Aplikácia obsahuje štyri obrazovky implementované ako Jetpack Compose composable funkcie. Každá obrazovka rieši unikátnu funkciu a je zobrazená na celom displeji zariadenia.

6.2 ViewModel a Lifecycle

Každá obrazovka má vlastný ViewModel anotovaný @HiltViewModel, ktorý drží stav obrazovky ako StateFlow<UiState>. ViewModely prežívajú zmenu konfigurácie zariadenia (rotácia displeja) - filter kategórie, vyhľadávací text a GPS súradnice zostanú zachované. Vo všetkých obrazovkách sa používa funkcia collectAsStateWithLifecycle() namiesto collectAsState(), čo zabezpečuje, že Flow sa prestane zbierať keď je používateľské rozhranie na pozadí, a tým sa šetrí zdroje zariadenia.

6.3 Navigation

Navigácia medzi obrazovkami je implementovaná pomocou NavHost a NavController z knižnice Navigation Compose. Každá obrazovka má definovanú zapečatenú triedu NavRoutes. Obrazovka DetailScreen prijíma identifikátor miesta placeId ako navigačný argument (route = "detail/{placeId}"). Spodná navigačná lišta je implementovaná ako NavigationBar s položkami NavigationBarItem, pričom stav aktívnej záložky je odvodený z aktuálneho záznamu v zásobníku.

6.4 WorkManager a notifikácie

Trieda RefreshWorker implementuje CoroutineWorker a je anotovaná @HiltWorker pre podporu injekcie závislostí. Worker je naplánovaný ako periodická úloha s intervalom 6 hodín pomocou PeriodicWorkRequestBuilder s podmienkou NetworkType.CONNECTED. Na zamedzenie duplicitného spustenia sa používa politika ExistingPeriodicWorkPolicy.KEEP. Po úspešnom refreshi je používateľovi zobrazená notifikácia prostredníctvom triedy RefreshNotificationHelper. Notifikačný kanál je vytvorený s úrovňou dôležitosti IMPORTANCE_LOW.

6.5 Sieťová komunikácia - Retrofit a Overpass API

Aplikácia komunikuje s Overpass API (OpenStreetMap) pomocou knižnice Retrofit 2 s konvertorom Moshi. Rozhranie OverpassApiService definuje GET endpoint s parametrom @Query("data") pre Overpass QL dopyt. Dopyt načítava všetky POI kategórie v ohraničujúcom obdĺžniku mesta Žilina. Pri nedostupnosti siete sa automaticky načítajú záložné údaje zo statického súboru res/raw/places_fallback.json, ktorý obsahuje 70 ručne vybraných miest. Logika prepínania je implementovaná v metóde PlacesRepositoryImpl.refreshPlaces().

6.6 GPS senzor

Aplikácia využíva GPS senzor zariadenia prostredníctvom triedy LocationHelper, ktorá používa Fused Location Provider API (FusedLocationProviderClient). Implementácia je obalená do suspend funkcie pomocou suspendCancellableCoroutine. GPS sa využíva na dvoch miestach: vycentrovanie mapy na aktuálnu polohu používateľa (tlačidlo FAB v MapScreen) a zoradenie zoznamu miest podľa vzdialenosti (metóda ListViewModel.onSortToggled()). Vzdialenosť sa vypočítava pomocou Haversineho vzorca implementovaného v metóde



Place.distanceTo().

6.7 DataStore - Obľúbené miesta

Obľúbené miesta sú perzistované pomocou DataStore Preferences bez potreby relačnej databázy. Trieda FavoritesDataStore ukladá iba množinu identifikátorov Set<String> obľúbených miest. Všetky operácie sú bezpečné z hľadiska koroutín. Používateľské rozhranie automaticky reaguje na zmeny prostredníctvom Flow<Set<String>> exponovaného z FavoritesDataStore.

6.8 OSMDroid - Mapa

OSMDroid je open-source knižnica pre zobrazenie máp OpenStreetMap na zariadeniach Android bez potreby API kľúča. V aplikácii je integrovaná ako wrapper komponent AndroidView (súbor OsmMapView.kt). Markery sú pridávané dynamicky pomocou objektov Marker a po kliknutí presmerujú používateľa na DetailScreen. Vrstva MyLocationNewOverlay zobrazuje aktuálnu polohu používateľa formou modrého bodu na mape.

6.9 Zachovávanie stavu pri rotácii

Stav aplikácie prežíva rotáciu displeja na viacerých úrovniach. ViewModel automaticky prežíva zmenu konfigurácie zariadenia. Funkcia rememberSaveable sa používa pre lokálny stav obrazovky, napríklad pre uchovanie informácie o udelení povolenia na GPS (locationGranted). Inštancia MapView pre OSMDroid je vytvorená v bloku remember, čím prežíva rekompozície, a jej životný cyklus je spravovaný pomocou DisposableEffect.

6.10 Architektúra a kvalita kódu

Aplikácia dodržiava architektúru MVVM so vzorom Repository. UI vrstva závisí iba na ViewModeloch, ViewModely závisí na rozhraní PlacesRepository - nie na konkrétnej implementácii. Injekciu závislostí zabezpečuje Hilt. Všetky sieťové a vstupno-výstupné operácie prebiehajú v Kotlin koroutínach mimo hlavného vlákna. Každá verejná trieda a funkcia je zdokumentovaná pomocou KDoc komentárov. Kód je organizovaný do balíčkov podľa funkcie: data, di, model, repository, ui, worker.

7. Zoznam použitých zdrojov

1. Android Developers - Jetpack Compose: <https://developer.android.com/jetpack/compose>
2. Android Developers - ViewModel:



<https://developer.android.com/topic/libraries/architecture/viewmodel>

3. Android Developers - Navigation Compose:

<https://developer.android.com/jetpack/compose/navigation>

4. Android Developers - WorkManager:

<https://developer.android.com/topic/libraries/architecture/workmanager>

5. Android Developers - DataStore:

<https://developer.android.com/topic/libraries/architecture/datastore>

6. Android Developers - Hilt: <https://developer.android.com/training/dependency-injection/hilt-android>

7. Retrofit 2: <https://square.github.io/retrofit/>

8. Moshi: <https://github.com/square/moshi>

9. OSMDroid: <https://github.com/osmdroid/osmdroid/wiki>

10. Overpass API: https://wiki.openstreetmap.org/wiki/Overpass_API

11. Coil: <https://coil-kt.github.io/coil/compose/>

12. Material 3: <https://m3.material.io/components>

13. Kotlin Coroutines Flow: <https://kotlinlang.org/docs/flow.html>

14. Haversine formula: <https://www.movable-type.co.uk/scripts/latlong.html>

15. OpenStreetMap: <https://www.openstreetmap.org>